



Universidad EAFIT

Estructuras de Datos y Algoritmos

Análisis de redes de transporte reales

Practica Final Integradora

Presentado por:

Tomás Noreña Toro

Miguel Muñoz Jiménez

Profesor:

Carlos Alberto Álvarez Henao

1.Introducción

Para esta práctica final, trabajamos con la red vial del estado de Pennsylvania en Estados Unidos. El objetivo era aplicar los algoritmos y conceptos que vimos durante el semestre a una escala mas grande, un grafo con más de un millón de nodos, mas cercano a la realidad.

El conjunto de datos que usamos de llama roadNet-PA y es parte de la colección SNAP de Stanford. Este mismo dataset fue usado en el 9th DIMACS Implementación Challenge del año 2006, que es una competencia importante a nivel internacional donde grupos de investigación compiten implementando algoritmos de camino más corto. Usar ese dataset nos permite verificar nuestros resultados contra valores ya publicados.

En este grafo, cada nodo representa una intersección vial y cada arista representa un tramo de carretera. Tiene 1,088,092 nodos y 1,541,898 aristas. La razón por la que esto es relevante en la práctica es que aplicaciones como Google maps o Waze usan exactamente este tipo de algoritmos para calcular rutas, solo que con grafos aún mas grandes y con pesos que representan tiempos de viaje reales.

Los algoritmos que implementamos fueron: BFS para recorrer el grafo y estimar el diámetro, Dijkstra con heap para calcular caminos mínimos con pesos, Kruskal para el árbol de expansión mínima y DFS para verificar si el subgrafo es un DAG.

2. Descripción del dataset

El archivo roadNet-PA.txt tiene una arista por línea, donde cada línea tiene dos números separados por un tabulador. Las líneas que empiezan con “#” son comentarios y no se cargan. Originalmente el grafo es dirigido, pero lo tratamos como no dirigido insertando ambas direcciones de cada arista (u->v y v->u).

Estos fueron los resultados que obtuvimos al cargar el grafo, comparados con los valores que publica SNAP:

Estadística	Valor SNAP	Valor obtenido	¿Coinciden?
Nodos totales	1,088,092	1,088,092	Sí
Aristas (No dirigidas)	1,541,898	1,541,898	Sí
Nodos en componente principal	1,087,562	1,087,562	Sí
Grado promedio	~2.83	~2.83413	Sí
Diámetro	782	592	No, lo nuestro es una aproximación

El único valor que no coincidió exactamente fue el diámetro. Nosotros obtuvimos 592 y SNAP 782. Esto pasa porque calculamos el diámetro haciendo BFS desde el nodo con más conexiones, y este método solo da una estimación. Para encontrar el diámetro exacto habría que hacer BFS desde todos los nodos, lo cual tomaría demasiado tiempo en un grafo de este tamaño.

Una cosa interesante del grafo es que es muy disperso, cada nodo en promedio tiene menos de tres conexiones. Eso tiene sentido para una red vial porque la mayoría de intersecciones conectan con pocas calles.

3. Diseño e implementación:

3.1 Como representamos el grafo

Usamos la lista de adyacencia para guardar el grafo en memoria. En C++ esto se implementa con `vector<vector<pair<int,int>>>`, donde cada posición del vector exterior representa un nodo y su vector interno tiene los pares(nodo vecino, peso de la arista).

No usamos matriz de adyacencia porque requeriría una cantidad absurda de memoria. Exactamente $1,088,092 \times 1,088,092$ (cantidad de celdas) $\times$ 4 bytes (una celda guarda un entero de 32 bits = 4 bytes) = sería aproximadamente 4.7 TB. Esa cantidad de memoria no la tiene ningún computador normal. Mientras que con lista de adyacencia el grafo sería mucho más liviano, ocupando entre 50 y 80 MB ya que solo guardamos las aristas que existen, no todas las posibles, además tenemos 1,541,898 aristas, pero como es no dirigido son 3,083,796 entradas, cada entrada tiene un par (nodo vecino, peso) = 2 enteros = 8 bytes, lo que nos daría $3,083,796 \times 8 = 24,670,368$ bytes $\approx$ 24 MB solo para las aristas, Sumando el vector exterior y otros datos internos de C++, queda entre 50 y 80 MB en total.

3.2 Los pesos de las aristas

EL dataset no trae pesos, así que los generamos nosotros de forma aleatoria entre 1 y 10 usando `srand(42)` antes de leer el archivo. La semilla fija es importante porque garantiza que si otro equipo corre el mismo código va a obtener exactamente los mismos pesos y por lo tanto los mismos resultados en las consultas.

3.3 Como funciona Dijkstra en nuestro código

El algoritmo de Dijkstra que implementamos usa una cola de prioridad (heap mínimo) para siempre procesar primero el nodo más cercano al origen. Además de calcular la distancia mínima, guardamos en un arreglo `prev[]` el nodo anterior en el camino más corto, lo que nos permite reconstruir la ruta completa al final.

Los pasos básicos son:

- Empezar con distancia 0 para el nodo de origen y distancia infinita para todos los demás.
- Meter el nodo origen en el heap con distancia 0
- Mientras el heap no esté vacío, sacar el nodo con menor distancia.
- Para cada vecino de ese nodo, si encontramos un camino mas corto, actualizamos su distancia y lo metemos al heap.
- Parar cuando lleguemos al nodo destino.
- Reconstruir el camino siguiendo el arreglo `prev[]` desde el destino hasta el origen.

3.4 Como estimamos el diámetro con BFS

para estimar el diámetro del grafo hicimos BFS desde el nodo con mayor grado. BFS recorre el grafo nivel por nivel y la distancia máxima que encuentra es nuestra estimación del diámetro. No es el diámetro exacto, pero es una aproximación razonable y mucho más rápida de calcular.

4. Análisis de complejidad

4.1 Dijkstra con heap $O((V+E) \log V)$

Dijkstra con heap mínimo tiene una complejidad de $O((V + E) \log V)$, donde V es el número de nodos y E el número de aristas (edges en inglés). Esto se debe a que cada nodo puede salir del heap una vez ($O(\log V)$ por operación, $O(V \log V)$ en total), y cada arista puede generar una inserción al heap ($O(\log V)$ por inserción, $O(E \log V)$ en total).

Para nuestro grafo: $(1,088,092 + 1,541,898) \times \log(1,088,092) \approx 2,630,000 \times 20 \approx 52$ millones de operaciones. En la práctica vimos tiempos de entre 5 ms y 177 ms por consulta, que es bastante razonable para esa cantidad de operaciones.

4.2 BFS: $O(V+E)$

BFS es más simple: visita cada nodo exactamente una vez y examina cada arista exactamente una vez, así que su complejidad es de $O(V+E)$. Para nuestro grafo eso son 2.6 millones de operaciones, lo que explica porque BFS es entre 4 y 10 veces más rápido que Dijkstra en nuestras pruebas.

4.3 Por que no podemos usar una matriz de adyacencia

Con matriz de adyacencia, Dijkstra tendría complejidad $O(V^2)$ porque para encontrar los vecinos de un nodo hay que recorrer toda una fila de la matriz. Para $V = 1,088,092$ eso sería mas de un billón de operaciones, lo cual es inviable en tiempo además de en memoria. La lista de adyacencia nos permite aprovechar que el grafo es muy disperso (pocas aristas por nodo), haciendo el algoritmo mucho más eficiente.

5. Resultados de las consultas P2P

Corrimos las 10 consultas fijas del enunciado con Dijkstra y con BFS. Estos fueron los resultados obtenidos:

Consulta	Origen	Destino	Dist. Dijkstra	Salto BFS	Nodos Dijk.	Nodos BFS	T. Dijk. (ms)	T. BFS (ms)
Q01	1	500000	516	101	69,880	54,793	8.14	1.33
Q02	100	1000000	2304	484	1,106,267	1,008,992	164.93	36.31
Q03	50000	750000	2702	545	1,040,092	923,563	143.31	31.17
Q04	200000	800000	612	124	106,905	88,371	13.54	2.59
Q05	300000	100000	3170	666	1,174,348	1,052,631	160.70	34.34
Q06	1	1087562	437	95	48,428	47,313	4.88	1.03
Q07	500000	1	516	101	102,434	91,333	13.81	2.93
Q08	250000	600000	1433	300	813,824	758,734	127.29	26.93
Q09	10000	900000	844	169	194,812	156,502	25.65	5.07
Q10	400000	150000	859	167	431,720	352,068	65.68	14.72

- Lo que más nos llamó la atención de los resultados fue: BFS siempre fue más rápido que Dijkstra, entre 4 y 10 veces. Eso tiene sentido porque BFS es más simple y no tiene que manejar pesos ni el heap.
- Las consultas más rápidas fueron Q01 y Q06, que salían del nodo 1. Eso sugiere que el nodo 1 está en una zona bien conectada del grafo.
- Q05 fue la más lenta porque Dijkstra tuvo que explorar más de un millón de nodos antes de encontrar el camino.
- Q01 y Q07 son la misma ruta pero al revés (nodo 1 a 500000, y nodo 500000 a 1). Tienen la misma distancia (516) y los mismos saltos (101), pero Dijkstra exploró más nodos en Q07. Eso pasa porque Dijkstra no es simétrico en cuántos nodos explora.
- Ninguna consulta devolvió distancia infinita, lo que confirma que todos los nodos del dataset están conectados entre sí a través de la componente principal.

6. Análisis del subgrafo

El subgrafo se construyó tomando todos los nodos que aparecen en los caminos de Q01 (del nodo 1 al 500000) y Q06 (del nodo 1 nodo 108756). Como los dos caminos comparten los primeros nodos (1, 0, 6309, 6310...), el subgrafo no es la simple suma de los dos caminos sino la unión de sus nodos únicos.

Propiedad	Valor
Nodos del subgrafo	216
Aristas del subgrafo	218
Peso total del MST (Kruskal)	936
Aristas en el MST	215
Es DAG	No

El MST calculado con Kruskal tiene 215 aristas para 216 nodos. Eso es correcto porque un árbol de expansión mínima sobre n nodos siempre tiene exactamente $n-1$ aristas.

El subgrafo no es un DAG, esto se podía ver desde antes ya que el grafo es no dirigido. En un grafo no dirigido, cualquier arista entre dos nodos crea automáticamente un ciclo (poder ir de u a v y volver de v a u), así que nunca puede ser un DAG. Aun así, lo verificamos con DFS como se solicitaba.

Si pensamos en lo que representa este subgrafo geográficamente, son los caminos viales que conectan dos rutas distintas que salen del mismo punto (nodo 1) y llegan a dos destinos distintos en Pennsylvania. Los primeros nodos que comparten ambos caminos serían las calles iniciales por las que salen ambas rutas antes de separarse hacia destinos distintos.

7. Conclusiones

Este proyecto fue un gran acercamiento a un caso de la realidad donde se usen estos conceptos. Trabajamos con un grafo de verdad, mucho mas grande a los que solemos ver en la teoría, nos mostró varias cosas que no se ven en ejemplos pequeños:

- Los tiempos de ejecución si importan. En los ejemplos teóricos los algoritmos terminan en milisegundos sin importar como fueron implementados, pero aquí la diferencia entre BFS y Dijkstra fue muy clara: BFS era hasta 10 veces más rápido en algunas consultas.
- La elección de la estructura de datos es crítica. Si hubiéramos intentado usar una matriz de adyacencia, el programa simplemente no hubiera podido correr por falta de memoria. La lista de adyacencia fue la única opción viable.

- Estimar el diámetro exacto de un grafo grande es muy difícil. Nuestra estimación dio 592 cuando el valor real es 782 y eso fue haciendo BFS desde un solo nodo. Calcular el diámetro exacto es mucho mas costoso en tiempo y recursos.
- La reproducibilidad con srand(42) funcionó perfectamente. Cada vez que corrimos el programa obtuvimos los mismos resultados, lo que es importante para que estos sean verificables.

Una limitación de nuestro trabajo es que los pesos de las aristas son aleatorios entre 1 y 10 y no representan la distancia en metros o el tiempo real. En un sistema de navegación real, cambiaría completamente que rutas son las más cortas.

También notamos que el array del Union-Find (algoritmo de Kruskal) en el Módulo C tiene tamaño fijo de 2000, lo que funcionó para este subgrafo de 216 nodos, pero podría fallar con subgrafos grandes. Es algo que mejoraríamos si tuviésemos tiempo

8. Referencias

9th DIMACS Implementation Challenge: Shortest Paths. (s. f.).

<https://www.diag.uniroma1.it/~challenge9/>

Stanford large network dataset collection. (s. f.). <http://snap.stanford.edu/data/>

Material del curso Estructuras de Datos y Algoritmos 2026-1, Universidad EAFIT.

Herramientas usadas: Claude (Anthropic) y Google Gemini fueron utilizadas como apoyo durante el desarrollo del proyecto. Se utilizaron para resolver dudas sobre errores de compilación, porque aparecían errores de definiciones múltiples al enlazar archivos como los siguientes: multiple definition of `reconstruirCamino` multiple definition of `dijkstra` multiple definition of `bfs`, entender el funcionamiento de GitHub y algunas complicaciones de push y pull que llegamos a tener. Todo el código que se consultó fue revisado, probado y comprendido por los integrantes del equipo antes de incluirlo en el repositorio.